

50. Une vitesse qui monte

Accélérer, c'est une variable de plus.

1. Ce qu'il faut comprendre

Un objet qui accélère, c'est une **vitesse** qui monte. Tout ce qui suit — le saut, la pesanteur — n'est que cela, avec un signe en plus.

Elle ne montera pourtant **jamais jusqu'à 5** : plus le caillou va vite, plus tôt il touche le bas, et la chute suivante repart à 1. Le plafond écrit dans le code n'est pas celui qu'on observe — c'est le genre d'écart qu'on ne voit qu'en faisant tourner.

Pourquoi `depuis` et non `images() % 20 == 0`. Les deux paraissent équivalents, et ils ne le sont pas. Tester un multiple suppose que la boucle tourne **exactement une fois par image** ; dès que le corps s'alourdit, un tour franchit deux images d'un coup, le multiple est enjambé, et l'accélération n'arrive jamais.

Un compteur incrémenté à chaque tour et comparé avec `>=` ne peut pas rater son échéance. C'est la forme à prendre par défaut.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <sprite>` → [0.69.4. La fonction `sprite\(\)` — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **279 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      47 o ← la console, à l'allumage - la mise en route de la console
main()         97 o ← démarrage - ta fonction
VBlank         8 o  ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
RangerLutins   11 o ← démarrage - range les 40 lutins hors de l'écran
InstallerTransfert 15 o ← démarrage - installe la copie des lutins en mémoire rapide
CopierTuiles   31 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
Tuiles         16 o ← CopierTuiles - 0 lettre sur 43, et 1 tuile dessinée
DonneesTransfert 10 o ← InstallerTransfert - la petite routine qui copie les lutins
```

Lettres gravées : aucune

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `EffacerCarte`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `Notes`

Fonctions du langage appelées :

```
sprite() × 1    23 o sur place, dans main()
image() × 1     6 o sur place, dans main() + AttendreImage (28 o)
```

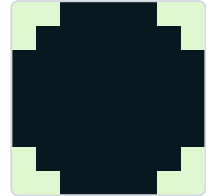
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : un caillou tombe de plus en plus vite, puis recommence en haut.
//
// La vitesse est une variable : le nombre de pixels parcourus à chaque image.
// Toutes les 20 images, la vitesse augmente de 1 (jusqu'à 5).

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <sprite>   // place un lutin de 8 × 8 au pixel près
```

2. UN DESSIN — CAILLOU

```
Tuile CAILLOU = {
    "..####..", ".#####.", "#####", "#####",
    "#####", "#####", ".#####.", "..####..",
};
```



à l'écran

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                // Le jeu commence ici.
    uint8_t y = 0;          // La hauteur du caillou, en pixels (0 = tout en haut).
    uint8_t vitesse = 1;    // Les pixels descendus à chaque image.
    uint8_t depuis = 0;     // Les images depuis la dernière accélération.

    while (true) {          // Répète pour toujours :
        y += vitesse;       // « y += vitesse » = « y = y + vitesse » : il descend.
        if (y > 130) { y = 0; vitesse = 1; depuis = 0; }
                               // Arrivé en bas : on remet tout au départ.

        depuis++;           // Une image de plus.
        if (depuis >= 20) { depuis = 0; if (vitesse < 5) vitesse++; }
                               // Toutes les 20 images : plus vite (sans dépasser 5).

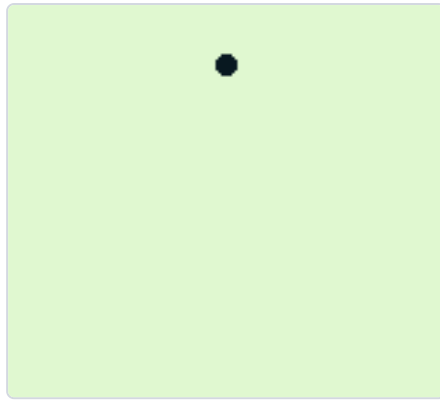
        sprite(0, 76, y, CAILLOU); // On repose le caillou.
        image();              // Attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

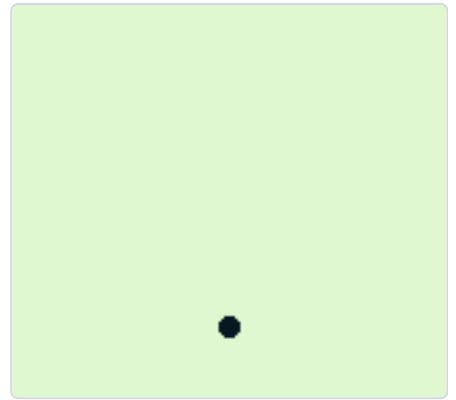
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



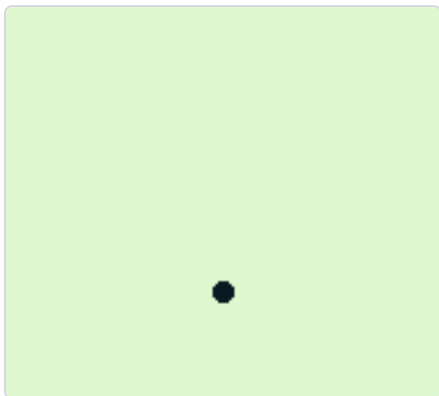
la toute première image — la console vient de démarrer — image 1



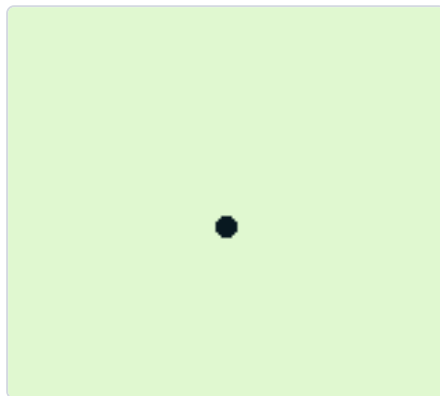
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un caillou qui tombe de plus en plus vite, et recommence.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la vitesse monte (jusqu'à 4)
- ✓ sans dépasser le plafond écrit
- ✓ le caillou parcourt tout l'écran (y jusqu'à 128)
- ✓ et il recommence en haut

Ensuite : **51. Sauter, sans pesanteur** — chapitre 7 : Le mouvement. · [gameboy3](#) — écrire une cartouche Game Boy en C++